הנחיות לכתיבת תוכנה מקצועית ברמת הצטיינות יתרה

Guidelines for Writing Professional Software at the Highest Level of Excellence

---

ד"ר יורם סגל

כל הזכויות שמורות - © ד"ר יורם סגל

2026-03-26

גרסה 3.00

תוכן העניינים

1 מבוא — המתכנת המקצועי בעידן הבינה המלאכותית 5
1.1 מהו מתכנת מקצועי? 5
2.1 עבודת צוות בפיתוח תוכנה 5
3.1 מחזור חיי פרויקט תוכנה 5
4.1 המהפכה: בינה מלאכותית וקידוד בהנחיה 6
2 מבנה פרויקט חובה ומסמכי תיעוד 7
1.2 קובץ README.md בשורש הפרויקט 7
2.2 תיקיית docs/ — מסמכי תיעוד חובה 7
3.2 מסמכי PRD ייעודיים לאלגוריתמים ומנגנונים 8
4.2 מבנה פרויקט מומלץ 8
5.2 תהליך עבודה חובה 9
3 תיעוד קוד ומבנה פרויקט 01
1.3 מבנה פרויקט מודולרי 01
2.3 כלל גודל הקבצים — מקסימום 150 שורות 01
3.3 איכות קוד והערות 01
4 ארכיטקטורת SDK ועיצוב מונחה-עצמים 11
1.4 ארכיטקטורה מבוססת SDK 11
2.4 עיצוב מונחה-עצמים (OOP) — ללא כפילות קוד 11
5 שומר סף API ובקרת קצב 31
1.5 שומר סף מרכזי (API Gatekeeper) 31
2.5 קונפיגורציית הגבלת קצב 31
3.5 ניהול תורות לגלישה 41
6 פיתוח מונחה-בדיקות ואבטחת איכות 51
1.6 תהליך TDD — אדום, ירוק, שיפור 51
2.6 כיסוי בדיקות — מינימום 85% 51
3.6 טיפול במקרי קיצון ותקלות 61
4.6 תוצאות בדיקה צפויות 61
7 בדיקת קוד, ניהול קונפיגורציה ואבטחת מידע 71
1.7 עמידה ב-Linter — אפס הפרות Ruff 71
2.7 איסור ערכים מוטבעים בקוד 71
3.7 ארכיטקטורת קונפיגורציה 81
4.7 אבטחת מידע וניהול סודות 81
8 בקרת גרסאות ומנהל חבילות uv 91

1.8 בקרת גרסאות גלובלית 91
2.8 שיטות עבודה מומלצות עם Git 91
3.8 ספר הפרומפטים 91
4.8 מנהל חבילות uv — חובה 91
9 מחקר וניתוח תוצאות 12
1.9 חקר פרמטרים 12
2.9 מחברת ניתוח תוצאות 12
3.9 הצגה ויזואלית של תוצאות 12
10 ממשק משתמש וחווית משתמש 22
1.10 קריטריוני איכות 22
2.10 תיעוד ממשק 22
11 עלויות ותמחור 32
1.11 ניתוח עלויות 32
2.11 ניהול תקציב 32
12 הרחבה ותחזוקתיות 42
1.12 נקודות הרחבה 42
2.12 תחזוקתיות 42
13 תקני איכות בינלאומיים 52
1.13 מאפייני איכות מוצר 52
14 ארגון הפרויקט כחבילה 62
1.14 קובץ הגדרת החבילה 62
2.14 קבצי `__init__.py` 62
3.14 נתיבים יחסיים 62
4.14 רשימת בדיקה: ארגון כחבילה 62
15 עיבוד מקבילי וביצועים 72
1.15 הבדל בין Multiprocessing ל-Multithreading 72
2.15 בטיחות חוטים 72
3.15 רשימת בדיקה: עיבוד מקבילי 72
16 עיצוב מודולרי ואבני בניין 82
1.16 מבנה אבן בניה 82
2.16 עקרונות עיצוב 82
3.16 דוגמה לאבן בניה 82
17 רשימת בדיקה סופית 03
1.17 מבנה ותיעוד חובה 03

כל הזכויות שמורות - © Dr. Segal Yoram

2.17 ארכיטקטורה וקוד 03
3.17 בדיקות ואיכות 03
4.17 קונפיגורציה ואבטחה 03
5.17 מחקר וויזואליזציה 13
6.17 הרחבה ותקנים 13
18 מקורות ותקנים נוספים 23
19 הערה חשובה 33
1.19 כרטיס עזר מהיר לדרישות 33
English References34
20 נספח: הנחיות מפורטות להגשת תוכנה מקצועית 63
1.20 מסמכי פרויקט ותכנון 63
2.20 תיעוד קוד ומבנה פרויקט 63
3.20 ניהול קונפיגורציה ואבטחת מידע 73
4.20 בדיקות ואיכות תוכנה 73
5.20 מחקר וניתוח 83
6.20 ממשק משתמש 83
7.20 ניהול גרסאות ועלויות 83
8.20 הרחבה ותקנים 93
9.20 רשימת בדיקה סופית 93

1 מבוא — המתכנת המקצועי בעידן הבינה המלאכותית

1.1 מהו מתכנת מקצועי?

מתכנת מקצועי (Professional Software Engineer) הוא לא רק מי שיודע לכתוב קוד, אלא מי שמבין את מלוא מחזור החיים של תוכנה, יודע לתכנן, לתעד, לבדוק ולתחזק מערכות מורכבות. מתכנת מקצועי עומד בסטנדרטים גבוהים של איכות קוד, מקפיד על שיטות עבודה מוכחות (best practices), ומסוגל לעבוד בצוות תוך שמירה על עקביות, תקשורת ברורה ואחריות.

המאפיינים המרכזיים של מתכנת מקצועי כוללים:

- **חשיבה מערכתית** — היכולת לראות את התמונה הגדולה ולהבין כיצד כל רכיב משתלב במערכת כולה
- **תכנון לפני ביצוע** — כתיבת מסמכי דרישות, ארכיטקטורה ותכנון לפני שורת קוד ראשונה
- **איכות ללא פשרות** — הקפדה על כתיבת קוד נקי, מתועד, מכוסה בבדיקות ובטוח
- **למידה מתמדת** — עדכון ידע בטכנולוגיות, כלים ומתודולוגיות חדשות
- **תקשורת אפקטיבית** — יכולת לתאר פתרונות טכניים לקהלים מגוונים

1.2 עבודת צוות בפיתוח תוכנה

פיתוח תוכנה מקצועי מתבצע כמעט תמיד בצוות. עבודת צוות אפקטיבית מבוססת על מספר עקרונות מפתח:

תפקידים ואחריות — צוות פיתוח טיפוסי כולל ארכיטקט תוכנה (Software Architect) האחראי על תכנון המערכת, מפתחים (Developers) המיישמים את הקוד, מהנדס בדיקות (QA Engineer) המוודא את איכות המוצר, מנהל מוצר (Product Manager) המגדיר את הדרישות, ולעיתים גם מפתח DevOps האחראי על תשתיות ותהליכי פריסה.

תהליכי עבודה — צוותים מקצועיים עובדים עם מתודולוגיות כגון Scrum, Kanban או SAFe, הכוללות ישיבות יומיות (daily standup), סקירות ספרינט (sprint reviews), רטרוספקטיבות, וסקירות קוד (code reviews) הדדיות באמצעות Pull Requests.

סטנדרטים משותפים — הצוות מגדיר ומקפיד על תקני קוד (coding standards), מבנה פרויקט אחיד, ותהליכי בדיקה ופריסה אוטומטיים (CI/CD).

1.3 מחזור חיי פרויקט תוכנה

כל פרויקט תוכנה מקצועי עובר מחזור חיים מוגדר (Software Development Life Cycle – SDLC):

1. **הגדרת דרישות** — כתיבת מסמך דרישות מוצר (PRD) מפורט
2. **תכנון וארכיטקטורה** — עיצוב המערכת, כתיבת מסמכי תכנון (PLAN) וקביעת אבני דרך (TODO)

כל הזכויות שמורות - Dr. Segal Yoram ©

3. **פיתוח** — כתיבת קוד לפי התכנון, בגישת TDD (Test-Driven Development)
4. **בדיקות** — בדיקות יחידה, אינטגרציה ומערכת
5. **פריסה** — הפצת התוכנה לסביבת הייצור
6. **תחזוקה ושיפור** — תיקון באגים, הוספת תכונות ושדרוגים

1.4 המהפכה: בינה מלאכותית וקידוד בהנחיה

בעידן הנוכחי, הבינה המלאכותית (AI) וסוכני AI משנים באופן מהותי את אופן פיתוח התוכנה. טכנולוגיית **קידוד בהנחיה** (Vibe Coding) — בה מתכנת מנחה סוכני AI ליצור, לבדוק ולשפר קוד — מאפשרת למתכנת בודד לתפקד כארכיטקט תוכנה בכיר (Senior Software Architect) המתזמר (orchestrate) מספר סוכני AI במקביל.

הכפלת כושר הייצור: מתכנת העובד עם סוכני AI ומשתמש בשיטת קידוד בהנחיה יכול לייצר בפרק זמן נתון פי 16 יותר שורות קוד איכותיות בהשוואה לכתיבה ידנית ללא AI. המשמעות היא שכל מתכנת, ללא קשר לוותק או ניסיון קודם, יכול להפוך למתכנת מקצועי ברמה גבוהה — בתנאי שהוא מקפיד על הכללים והתהליכים המוגדרים במסמך זה.

הכלל הראשון והחשוב ביותר: כדי לנצל את מלוא הפוטנציאל של סוכני AI, חובה להגדיר דרישות ברורות ומפורטות. ללא מסמכי דרישות, תכנון וארכיטקטורה — סוכני AI ייצרו קוד שאולי יעבוד, אבל לא יעמוד בסטנדרטים מקצועיים. לכן, **הכלל הראשון של קידוד מקצועי עם AI הוא להגדיר ולדרוש תיעוד מלא לפני כל שורת קוד**.

המסמך שלפניכם מגדיר את כל הדרישות, התקנים והתהליכים שיש לעמוד בהם כדי ליצור תוכנה מקצועית ברמה הגבוהה ביותר. עקבו אחר ההנחיות האלו, ותוכלו לנצל את מלוא הכוח של סוכני AI ולהפוך לארכיטקט תוכנה שמתזמר צוות של סוכנים — ומייצר תוצאות ברמה מקצועית שלא היו אפשריות בעבר.

2 מבנה פרויקט חובה ומסמכי תיעוד

כל פרויקט תוכנה מקצועי **חייב** לכלול מבנה תיקיות ומסמכי תיעוד מינימליים. ללא מסמכים אלו, הפרויקט לא ייחשב כעומד בדרישות המינימום.

2.1 קובץ README.md בשורש הפרויקט

כל פרויקט **חייב** לכלול קובץ `README.md` בתיקיית השורש (root directory). קובץ זה משמש כמדריך למשתמש ברמת user manual מלא[1], [2] וכולל:

- **הוראות התקנה** (Installation Instructions) — דרישות מערכת, התקנה שלב-אחר-שלב, הגדרת משתני סביבה ופתרון בעיות נפוצות
- **הוראות הפעלה** (Usage Instructions) — הרצה במצבים שונים, דגלים ואפשרויות CLI/GUI, ו-workflow טיפוסי
- **דוגמאות והדגמות** — דוגמאות קוד, צילומי מסך, תרחישי שימוש נפוצים
- **מדריך תצורה** (Configuration Guide) — הסבר על קבצי קונפיגורציה, פרמטרים והשפעתם
- **הנחיות תרומת קוד** (Contribution Guidelines) — תקני קוד וסגנון
- **רישיון וייחוס** (License & Credits) — רישיון שימוש וייחוס לספריות צד שלישי

2.2 תיקיית docs/ — מסמכי תיעוד חובה

כל פרויקט **חייב** לכלול תיקיית `docs/` בשורש הפרויקט, המכילה לפחות את המסמכים הבאים:

1. **מסמך דרישות מוצר —** `docs/PRD.md` (Product Requirements Document)

מסמך ה-PRD הוא המסמך המרכזי המגדיר את מטרת הפרויקט ואת הדרישות ממנו[3], [4], [5]. המסמך כולל:

- סקירת הפרויקט והקשר, תיאור בעיית המשתמש, ניתוח שוק וזיהוי קהל יעד
- יעדים מדידים, מדדי KPI וקריטריוני קבלה (acceptance criteria)
- דרישות פונקציונליות ולא-פונקציונליות, user stories ותרחישי שימוש
- הנחות, תלויות, מגבלות ופריטים מחוץ לתחום
- ציר זמן ואבני דרך עם משלוחים צפויים

2. **מסמך תכנון —** `docs/PLAN.md`

מסמך התכנון מתאר את הארכיטקטורה והתכנון הטכני של הפרויקט, וכולל:

- תרשימי C4 Model (Context, Container, Component, Code)
- תרשימי UML לתהליכים מורכבים ותרשימי deployment
- החלטות ארכיטקטוניות (ADRs) עם רציונל, trade-offs ואלטרנטיבות
- תיעוד API וממשקים, סכימות נתונים וקונטרקטים

3. **מסמך משימות —** `docs/TODO.md`

כל הזכויות שמורות - Dr. Segal Yoram ©

מסמך המשימות מפרט את כל המשימות הנדרשות ליישום הפרויקט:

- רשימת משימות מפורטת עם עדיפויות וסטטוס (טרם החל / בתהליך / הושלם)
- חלוקה לשלבים (phases) עם אבני דרך
- הקצאת אחריות לכל משימה
- קריטריוני סיום (definition of done) לכל משימה

2.3 מסמכי PRD ייעודיים לאלגוריתמים ומנגנונים

דרישה חשובה: לכל אלגוריתם ספציפי, מנגנון מרכזי, או רכיב טכני מורכב בפרויקט — יש ליצור מסמך PRD ייעודי ונפרד. לדוגמה:

- אלגוריתם למידת מכונה — `docs/PRD_ml_algorithm.md`
- מנגנון אימות משתמשים — `docs/PRD_authentication.md`
- מנוע חיפוש — `docs/PRD_search_engine.md`
- מערכת caching — `docs/PRD_caching.md`

כל מסמך PRD ייעודי כולל:

- תיאור מפורט של האלגוריתם או המנגנון, כולל רקע תיאורטי
- דרישות ספציפיות, קלט/פלט מצופה, ומדדי ביצוע
- אילוצים ומגבלות, חלופות שנשקלו, והסבר על הבחירה
- קריטריוני הצלחה ותרחישי בדיקה ספציפיים

2.4 מבנה פרויקט מומלץ

```
project-root/
├── src/                          # Source code
│   ├── <package>/
│   │   ├── __init__.py
│   │   ├── sdk/                  # SDK layer
│   │   │   └── sdk.py
│   │   ├── services/             # Business logic
│   │   ├── shared/               # Shared utilities
│   │   │   ├── gatekeeper.py     # API gatekeeper
│   │   │   ├── config.py         # Configuration manager
│   │   │   └── version.py        # Version tracking
│   │   └── constants.py
│   └── main.py
├── tests/                        # Unit and integration tests
│   ├── unit/
│   └── integration/
```

```
├── docs/                          # Documentation (MANDATORY)
│   ├── PRD.md                     # Product Requirements
│   ├── PLAN.md                    # Architecture & Planning
│   ├── TODO.md                    # Task tracking
│   └── PRD_<mechanism>.md         # Per-algorithm PRDs
├── config/                        # Configuration files
│   ├── setup.json
│   └── rate_limits.json
├── data/                          # Input data
├── results/                       # Experiment results
├── assets/                        # Images, graphs, resources
├── notebooks/                     # Analysis notebooks
├── README.md                      # MANDATORY
├── pyproject.toml                 # Build & dependencies
├── uv.lock                        # Locked dependencies
├── .env-example                   # Secret placeholders
└── .gitignore
```

2.5 תהליך עבודה חובה

תהליך העבודה המחייב הוא:

1. יצירת `docs/PRD.md` — ואישורו לפני המשך
2. יצירת `docs/PLAN.md` — תכנון ארכיטקטוני
3. יצירת `docs/TODO.md` — רשימת משימות
4. יצירת מסמכי PRD ייעודיים לכל אלגוריתם/מנגנון מרכזי
5. אישור כל המסמכים לפני תחילת הפיתוח
6. התחלת פיתוח — עדכון `TODO.md` עם התקדמות
7. שמירת תוצאות, יצירת ויזואליזציות, ועדכון `README.md`

כל הזכויות שמורות - Dr. Segal Yoram ©

3 תיעוד קוד ומבנה פרויקט

תיעוד קוד נכון ומבנה פרויקט מסודר הם יסודות חיוניים לפיתוח תוכנה מקצועי. תיעוד טוב מאפשר למפתחים אחרים להבין את הקוד במהירות, להשתמש בו בצורה נכונה, ולתרום לפרויקט ביעילות.

3.1 מבנה פרויקט מודולרי

ארגון נכון של מבנה הפרויקט הוא מפתח לתחזוקה יעילה ולהתפתחות עתידית של הקוד. עקרונות הארגון כוללים חלוקה לוגית של הפרויקט לתיקיות לפי תפקיד, כגון קוד מקור, בדיקות, תיעוד, נתונים, תוצאות, קונפיגורציה ומשאבים. הארגון יכול להיות מבוסס-תכונות (feature-based) או בארכיטקטורה שכבתית (layered architecture), תוך הפרדה ברורה בין קוד, נתונים, תוצאות ותיעוד.

3.2 כלל גודל הקבצים — מקסימום 150 שורות

כל קובץ קוד לא יעלה על 150 שורות קוד (שורות ריקות ושורות הערה לא נספרות). כאשר קובץ חורג מהמגבלה, **יש לפצל אותו למספר קבצים** — לעולם לא לדחוס קוד כדי להתאים למגבלה.

אסטרטגיות פיצול:

- **חילוץ פונקציות עזר** — פונקציות עצמאיות לקובץ נפרד
- **חילוץ mixin** — כאשר למחלקה מספר אחריויות
- **פיצול 50/50** — כאשר יש שתי חצאים לוגיים (קריאה/כתיבה)
- **חילוץ קבועים** — קבועים לקובץ `constants.py`
- **חילוץ מודלים** — הגדרות מודלים לקובץ נפרד

3.3 איכות קוד והערות

איכות הקוד נמדדת לא רק בפונקציונליות שלו אלא גם בקלות הקריאה והתחזוקה שלו. תקני הערות בקוד (Code Comments Standards)[6], [7], [8] דורשים שהערות יסבירו את ה-"למה" ולא רק את ה-"מה". כל פונקציה, מחלקה ומודול צריכים לכלול Docstrings מפורטים. ההערות צריכות להסביר החלטות עיצוב מורכבות, לתעד הנחות ותנאים מוקדמים, ולהתעדכן יחד עם שינויי הקוד.

עקרונות כתיבת קוד איכותי כוללים שימוש בשמות משתנים ופונקציות תיאוריים ומדויקים, כתיבת פונקציות קצרות וממוקדות העומדות בעיקרון האחריות היחידה (sin-gle responsibility), הימנעות מקוד כפול לפי עיקרון DRY (Don't Repeat Yourself), ושמירה על עקביות בסגנון הקוד לאורך כל הפרויקט.

4 ארכיטקטורת SDK ועיצוב מונחה-עצמים

4.1 ארכיטקטורה מבוססת SDK

כל פונקציה המכילה לוגיקה עסקית חייבת להיות נגישה דרך שכבת SDK. ה-SDK הוא נקודת הכניסה היחידה לכל הצרכנים: תפריטים, GUI, CLI, אינטגרציות עם צדדים שלישיים ושירותים עתידיים.

```
External Consumers (GUI / CLI / REST / Third Party)
        |
        v
   +---------+
   |   SDK   |  <-- Single entry point for ALL logic
   +----+----+
        |
        v
   +-------------+
   |   Domain    |  <-- Services, models, orchestrators
   |   Services  |
   +----+--------+
        |
        v
   +-----------------+
   | Infrastructure  |  <-- DB, file I/O, external APIs
   +-----------------+
```

דרישות ארכיטקטוניות:

- כל פונקציה עסקית חשופה דרך מחלקת SDK
- **אין לוגיקה עסקית** בשכבות GUI, CLI או בקרים — שכבות אלו מאצילות ל-SDK
- צרכנים חיצוניים יכולים לייבא את ה-SDK ולהפעיל את כל הפעולות ללא גישה למודולים פנימיים

4.2 עיצוב מונחה-עצמים (OOP) — ללא כפילות קוד

יש לתכנן קוד בגישת OOP. אין לכפול קוד. כאשר אותה לוגיקה מופיעה בשני קבצים או יותר, יש לחלץ אותה למודול משותף, מחלקת בסיס, או mixin.

- אותו גוף פונקציה בשני קבצים או יותר — חילוץ למודול משותף
- אותו דפוס `try/except` בשלושה קבצים או יותר — יצירת פונקציית עטיפה
- שיטה זהה בשלוש מחלקות או יותר — יצירת מחלקת בסיס או mixin
- לוגיקה מועתקת עם וריאציות קלות — שימוש בתבנית Template Method

כל הזכויות שמורות - Dr. Segal Yoram ©

כללי Mixin:

- כל mixin מספק דאגה אחת בלבד
- mixins לא דורסים שיטות אחד של השני
- mixins חייבים להיות ניתנים לבדיקה באופן עצמאי

5 שומר סף API ובקרת קצב

5.1 שומר סף מרכזי (API Gatekeeper)

כל קריאות API החיצוניות חייבות לעבור דרך שומר סף מרכזי. שומר הסף מטפל בהגבלת קצב, תורות, ניסיונות חוזרים וניטור.

דרישות שומר הסף:

- **אין קריאות API ישירות** שעוקפות את שומר הסף
- הגבלות קצב נאכפות לפני כל קריאה
- גלישה מועברת לתור (queue), לא נדחית
- כל קריאות API נרשמות לניטור

API Gatekeeper Interface

```
class ApiGatekeeper:
    """Centralized API call manager."""

    def __init__(self, config: RateLimitConfig):
        """Initialize with rate limit config."""
        ...

    def execute(self, api_call, *args, **kwargs):
        """Execute API call through gatekeeper.
        - Check rate limits before execution
        - Queue if limit reached
        - Retry on transient failures
        - Log all calls
        """
        ...

    def get_queue_status(self) -> QueueStatus:
        """Return queue depth and stats."""
        ...
```

5.2 קונפיגורציית הגבלת קצב

כל הגבלות הקצב חייבות להיקרא מקובץ קונפיגורציה, לעולם לא לקודד בקוד.

Rate Limit Configuration

```
{
  "rate_limits": {
```


כל הזכויות שמורות - Dr. Segal Yoram ©

```
    "version": "1.00",
    "services": {
      "default": {
        "requests_per_minute": 30,
        "requests_per_hour": 500,
        "concurrent_max": 5,
        "retry_after_seconds": 30,
        "max_retries": 3
      }
    }
  }
}
```

5.3 ניהול תורות לגלישה

כאשר הגבלות הקצב מגיעות לסף, שומר הסף **חייב** להעביר בקשות לתור במקום לדחות או לקרוס:

- תור FIFO לבקשות ממתינות
- עומק תור מקסימלי מוגדר בקונפיגורציה
- התראת לחץ (backpressure) כאשר התור מלא
- מנגנון ריקון שמעבד בקשות כאשר חלונות הקצב מתאפסים

6 פיתוח מונחה-בדיקות ואבטחת איכות

6.1 תהליך TDD — אדום, ירוק, שיפור

כל הפיתוח חייב לעקוב אחר Test-Driven Development: אדום (RED) — ירוק (GREEN) — שיפור (REFACTOR).

- כל מודול חייב לכלול קובץ בדיקות מתאים
- כל פונקציה/שיטה ציבורית חייבת לכלול לפחות בדיקה אחת
- הבדיקות מכסות הן את המסלול התקין והן מקרי שגיאה
- בדיקות נכתבות **לפני** או במקביל ליישום, לא כמחשבה שלאחר מעשה

מבנה בדיקות:

```
tests/
  unit/
    test_<module>/          # Mirror src/ structure
      test_<file>.py
  integration/
    test_<feature>.py
  conftest.py               # Shared fixtures
```

כללי בדיקות:

1. כל מודול חדש חייב קובץ בדיקות מתאים
2. כל פונקציה ציבורית חייבת לפחות בדיקה אחת
3. בדיקות מסלול תקין **וגם** מקרי שגיאה
4. שימוש ב-fixtures מ-`conftest.py` לנתוני בדיקה משותפים
5. Mock לתלויות חיצוניות (מסד נתונים, קבצים, API)
6. קבצי בדיקות עומדים גם הם בכלל 150 השורות
7. אין בדיקות שתלויות בשירותים חיצוניים

6.2 כיסוי בדיקות — מינימום 85%

כיסוי בדיקות גלובלי חייב להיות 85% ומעלה. חבילת הבדיקות חייבת להיכשל אם הכיסוי יורד מתחת לסף זה.

קונפיגורציה ב-`pyproject.toml`:

Coverage Configuration

```
[tool.coverage.run]
source = ["src"]
omit = ["src/main.py", "*/tests/*", "src/**/gui/*"]
```


כל הזכויות שמורות - Dr. Segal Yoram ©

```
[tool.coverage.report]
fail_under = 85
```

סוגי הכיסוי הנדרשים כוללים כיסוי הצהרות (Statement coverage), כיסוי ענפים (Branch coverage), וכיסוי מסלולים (Path coverage) למסלולים קריטיים.

6.3 טיפול במקרי קיצון ותקלות

זיהוי ותיעוד מקרי קיצון (Edge Cases) הוא חלק חיוני מפיתוח תוכנה איכותי. יש לזהות תנאי גבול, לתעד כל מקרה עם תיאור מפורט, ולכלול צילומי מסך של תקלות כאשר רלוונטי. מנגנוני טיפול בשגיאות חייבים לכלול תכנות הגנתי, הודעות שגיאה ברורות, רישום לוגים מפורט, והידרדרות חלקה (graceful degradation).

6.4 תוצאות בדיקה צפויות

יש לתעד את תוצאות ההרצה הצפויות לכל בדיקה, לייצר דוחות automated testing עם pass/fail rates, ולשמור לוגים של הרצות מוצלחות וכושלות.

7 בדיקת קוד, ניהול קונפיגורציה ואבטחת מידע

7.1 עמידה ב-Linter — אפס הפרות Ruff

אפס הפרות Ruff מותרות. כל הקוד חייב לעבור `ruff check` ללא שגיאות.

קונפיגורציה ב-`pyproject.toml`:

Ruff Configuration

```
[tool.ruff]
line-length = 100
target-version = "py310"

[tool.ruff.lint]
select = ["E","F","W","I","N","UP","B","C4","SIM"]
ignore = ["E501"]
```

קטגוריות כללים פעילים:

- **E** — שגיאות PEP 8 (הזחה, רווחים, סגנון)
- **F** — Pyflakes (שמות לא מוגדרים, ייבואים לא בשימוש)
- **W** — אזהרות PEP 8
- **I** — isort (סדר ייבואים)
- **N** — pep8-naming (מוסכמות שמות)
- **PU** — pyupgrade (מודרניזציה ל-Python 3.10+)
- **B** — flake8-bugbear (באגים נפוצים)
- **4C** — שימוש ב-comprehensions
- **MIS** — פישוט ביטויים

7.2 איסור ערכים מוטבעים בקוד

כל הערכים הניתנים לקונפיגורציה חייבים להגיע מקבצי קונפיגורציה, לא מקוד המקור.

טבלה 1: ערכים מוטבעים מול קונפיגורציה

קטגוריה	שימוש שגוי	שימוש נכון
כתובות API	”https://api.example.com”	cfg.get(”api_url”)
הגבלות קצב	rate_limit = 10	cfg.get(”rate_limit”, 10)
Timeouts	timeout=60	cfg.get(”timeout”, 60)
סודות	api_key = ”abc123”	os.environ.get(”API_KEY”)

ערכים מותרים בקוד: קבועים פיזיים/מתמטיים, ערכי ברירת מחדל לפרמטרים, קבועים

כל הזכויות שמורות - Dr. Segal Yoram ©

ב-constants.py, וערכי Enum.

7.3 ארכיטקטורת קונפיגורציה

כל הקונפיגורציה חייבת לעקוב אחר היררכיה ברורה עם קבצי קונפיגורציה מגורסנים.

```
config/
  setup.json                # Main app config (versioned)
  rate_limits.json          # API rate limits (versioned)
  logging_config.json       # Logging configuration
.env                        # Secrets (git-ignored)
.env-example                # Secret placeholders (committed)
pyproject.toml              # Build, lint, test settings
src/<package>/constants.py # Immutable project constants
```

7.4 אבטחת מידע וניהול סודות

אין נתונים סודיים בפרויקט. בעת דחיפה ל-GitHub, חובה ליצור `env-example`. עם ערכי דמה.

- אסור לשמור מפתחות API, סיסמאות או אסימונים בקוד המקור[9], [10], [11]
- שימוש אך ורק במשתני סביבה: `os.environ.get("API_KEY")`
- `.gitignore` חייב לכלול: `.env`, `*.pem`, `*.key`, `credentials.json`
- `env-example` חייב להיות קיים עם ערכי דמה
- בסביבות ייצור — שימוש בכלי ניהול סודות
- החלפת מפתחות תקופתית, ניטור שימוש, והגבלת הרשאות למינימום

8 בקרת גרסאות ומנהל חבילות uv

8.1 בקרת גרסאות גלובלית

הן הקוד והן קבצי הקונפיגורציה חייבים לכלול מעקב גרסאות מפורש. הגרסה מתחילה ב-1.00 ועולה בשינויים משמעותיים.

טבלה 2: מיקומי גרסאות נדרשים

ערך התחלתי	מיקום	פריט
1.00	src/<pkg>/shared/version.py	גרסת קוד
1.00	מפתח "version" ב-JSON	גרסת קונפיגורציה
1.00	"rate_limits.version"	גרסת הגבלות קצב

האפליקציה צריכה לבצע ולידציה של תאימות גרסת הקונפיגורציה בעת ההפעלה.

8.2 שיטות עבודה מומלצות עם Git

שיטות עבודה מומלצות כוללות שמירה על היסטוריית commits ברורה עם הודעות משמעותיות, שימוש ב-branches נפרדים לפיתוח תכונות חדשות, ביצוע סקירות קוד באמצעות Pull Requests, ושימוש ב-tagging לסימון גרסאות מרכזיות.

8.3 ספר הפרומפטים

תיעוד תהליך הפיתוח עם בינה מלאכותית (Prompt Engineering Log) כולל רשימת כל הפרומפטים המשמעותיים ששימשו לבניית הפרויקט, תיאור של ההקשר והמטרה, דוגמאות לפלטים שהתקבלו, שיפורים איטרטיביים, ושיטות עבודה מומלצות שהופקו מהניסיון.

8.4 מנהל חבילות uv — חובה

כל הפרויקטים חייבים להשתמש ב-uv כמנהל חבילות ומריץ משימות. אסור להשתמש ב-pip, pip install, python -m, venv או virtualenv ישירות.

טבלה 3: פקודות uv נדרשות

אסור	נכון (uv)	משימה
pip install	uv sync	התקנת תלויות
pip install <pkg>	uv add <pkg>	הוספת תלות
python script.py	uv run python script.py	הרצת סקריפט
python -m pytest	uv run pytest tests/	הרצת בדיקות
pip freeze	uv lock	נעילת תלויות

דרישות:

כל הזכויות שמורות - Dr. Segal Yoram ©

- `pyproject.toml` הוא מקור האמת היחיד לתלויות (ללא `requirements.txt`)
- `uv.lock` קיים ונמצא בבקרת גרסאות
- אין קריאות ישירות ל-pip או python -m בקוד, סקריפטים, CI/CD או תיעוד
- כל הכלים מופעלים דרך `uv run`

9 מחקר וניתוח תוצאות

מחקר מעמיק וניתוח תוצאות הם מה שמבדיל בין פרויקט תוכנה רגיל לבין עבודה מקצועית ברמת מצוינות. החלק המחקרי כולל ניסויים שיטתיים, ניתוח כמותי ואיכותי, והצגה ויזואלית של התוצאות.

9.1 חקר פרמטרים

ניתוח רגישות פרמטרים (Sensitivity Analysis) הוא תהליך שיטתי של בדיקת השפעת פרמטרים שונים על ביצועי המערכת. התהליך כולל ביצוע ניסויים שיטתיים עם שינוי מבוקר של פרמטרים, תיעוד מדויק של השפעת כל פרמטר, ושימוש בשיטות ניתוח מתקדמות כגון נגזרות חלקיות, ניתוח מבוסס שונות, או גישת "פרמטר-אחד-בכל-פעם" (OAT).

9.2 מחברת ניתוח תוצאות

מחברת ניתוח תוצאות (Results Analysis Notebook) היא כלי מרכזי להצגת המחקר. עומק הניתוח מושג באמצעות Jupyter Notebook או כלים דומים, ביצוע ניתוח מתודי של תוצאות הניסויים, השוואה בין אלגוריתמים, תצורות או גישות שונות, והכללת הוכחות מתמטיות או ניתוחים תיאורטיים. יש להשתמש ב-LaTeX לכתיבת משוואות ונוסחאות מקצועיות, ולכלול אסמכתאות לספרות אקדמית.

9.3 הצגה ויזואלית של תוצאות

ויזואליזציה איכותית של נתונים חיונית להעברת המסר המחקרי. סוגי הויזואליזציות כוללים Bar charts להשוואות, Line charts למגמות, Scatter plots למתאמים, Heatmaps לרגישות פרמטרים, Box plots להתפלגויות, ו-Waterfall charts לניתוח שינויים. איכות הגרפים נמדדת בבהירות התוויות, שימוש בצבעים עקביים ונגישים, כיתובים מפורטים ומקרא ברור, ורזולוציה גבוהה.

כל הזכויות שמורות - Dr. Segal Yoram ©

10 ממשק משתמש וחווית משתמש

ממשק משתמש (UI) וחווית משתמש (UX) טובים הם קריטיים להצלחת כל מערכת תוכנה[12], [13].

10.1 קריטריוני איכות

קריטריוני השימושיות כוללים: קלות למידה (Learnability), יעילות (Efficiency), זכירות (Memorability), מניעת שגיאות (Error Prevention), ושביעות רצון (Satisfaction).

עשרת ההיוריסטיקות של נילסן (Nielsen's 10 Heuristics)[14]: נראות סטטוס המערכת, התאמה בין המערכת והעולם האמיתי, שליטה וחופש למשתמש, עקביות ותקינה, מניעת שגיאות, זיהוי במקום זיכרון, גמישות ויעילות שימוש, עיצוב אסתטי ומינימליסטי, עזרה למשתמשים להתאושש משגיאות, ועזרה ותיעוד.

10.2 תיעוד ממשק

תיעוד מקיף של הממשק כולל צילומי מסך של כל מסך ומצב, תיאור workflow טיפוסי של משתמש, הסברים על אינטראקציות ופידבק, ושיקולי נגישות (accessibility).

11 עלויות ותמחור

הבנת עלויות הפיתוח והתפעול חיונית לתכנון נכון של הפרויקט.

11.1 ניתוח עלויות

ניתוח עלויות (Cost Breakdown) של שימוש ב-API Tokens כולל ספירה מדויקת של tokens בכניסה וביציאה, חישוב העלות למיליון tokens, והערכת העלות הכוללת לפי מודל ושירות.

טבלה 4: ניתוח עלויות API Tokens

Model / מודל	Input Tokens	Output Tokens	Total Cost / עלות כוללת
GPT-4	1,245,000	523,000	$45.67
Claude 3	890,000	412,000	$32.11
Total / סה"כ	2,135,000	935,000	$77.78

אסטרטגיות אופטימיזציה כוללות הפחתת שימוש ב-tokens, שימוש ב-batch processing, ובחירת מודלים לפי יחס עלות-תועלת.

11.2 ניהול תקציב

ניהול תקציב יעיל כולל תחזית עלויות לסקאלה, ניטור שימוש בזמן אמת, והגדרת התראות על חריגה מתקציב.

כל הזכויות שמורות - Dr. Segal Yoram ©

12 הרחבה ותחזוקתיות

12.1 נקודות הרחבה

ארכיטקטורת תוספים (Plugins Architecture) מאפשרת הוספת פונקציונליות חדשה ללא שינוי בקוד הליבה: ממשקים ברורים להרחבה, נקודות חיבור (lifecycle hooks) כגון beforeCreate, afterUpdate, מנגנוני middleware, ועיצוב מבוסס-API.

12.2 תחזוקתיות

קוד ניתן לתחזוקה מאופיין במודולריות והפרדת אחריות, שימוש חוזר של קומפוננטות, ניתנות לניתוח, וניתנות לבדיקה.

13 תקני איכות בינלאומיים

תקן ISO/IEC 25010[15] מגדיר מודל מקיף לאיכות תוכנה המכסה שמונה מאפייני איכות עיקריים.

13.1 מאפייני איכות מוצר

התאמה פונקציונלית (Functional Suitability) — שלמות, נכונות, והתאמה. יעילות ביצועים (Performance Efficiency) — זמני תגובה, ניצול משאבים, ויכולת. תאימות (Compatibility) — יכולת פעולה הדדית ודו-קיום. שימושיות (Usability) — קלות למידה, הפעלה, נגישות והגנה מפני שגיאות. אמינות (Reliability) — בשלות, זמינות, סובלנות לתקלות, ויכולת התאוששות. אבטחה (Security) — סודיות, שלמות, אימות, אחריותיות. תחזוקתיות (Maintainability) — מודולריות, שימוש חוזר, ניתנות לניתוח, לשינוי ולבדיקה. ניידות (Portability) — התאמה, ניתנות להתקנה, וניתנות להחלפה.

כל הזכויות שמורות - Dr. Segal Yoram ©

14 ארגון הפרויקט כחבילה

ארגון הקוד כחבילה (package) הוא עיקרון יסוד בפיתוח תוכנה מקצועי. חבילה מאורגנת נכון מאפשרת שימוש חוזר בקוד, ניהול תלויות ברור, הפצה והתקנה פשוטה, ובדיקות מובנות.

14.1 קובץ הגדרת החבילה

כל חבילה חייבת לכלול קובץ `pyproject.toml` (מועדף) או `setup.py` המפרט שם, גרסה, תיאור, מחבר, רישיון ותלויות.

14.2 קבצי `__init__.py`

קבצי `__init__.py` חייבים להימצא בתיקייה הראשית ובכל תת-תיקייה. מומלץ להשתמש בהם לייצוא ממשקים ציבוריים באמצעות `__all__` ולהגדרת `__version__`.

14.3 נתיבים יחסיים

כל הייבואים חייבים להשתמש בנתיבים יחסיים או בשמות חבילות, לעולם לא בנתיבים מוחלטים. גם קריאה/כתיבה של קבצים נעשית ביחס לנתיב החבילה.

14.4 רשימת בדיקה: ארגון כחבילה

1. **קובץ הגדרת חבילה:** האם קיים `pyproject.toml`? האם מכיל שם, גרסה ותלויות? האם התלויות מפורטות עם גרסאות?
2. **קובץ `__init__.py`:** האם קיים בתיקייה הראשית? האם מייצא ממשקים ציבוריים? האם `__version__` מוגדר?
3. **מבנה תיקיות:** האם קוד המקור בתיקייה ייעודית? האם הבדיקות ב-`tests/`? האם התיעוד ב-`docs/`?
4. **נתיבים יחסיים:** האם כל הייבואים יחסיים? האם נמנעים מנתיבים מוחלטים?

15 עיבוד מקבילי וביצועים

שימוש במעבדים מרובים (multiprocessing) ובחוטי ביצוע מרובים (multithreading) חיוני לביצועים אופטימליים של תוכנה מודרנית.

15.1 הבדל בין Multiprocessing ל-Multithreading

Multiprocessing מתאים לפעולות תובעניות מבחינת מעבד (CPU-bound): חישובים מתמטיים, עיבוד תמונות, אימון מודלים. כל תהליך פועל בזיכרון נפרד ומנצל ליבת מעבד שונה.

Multithreading מתאים לפעולות I/O-bound: קריאות רשת, גישה למסדי נתונים, קריאה/כתיבה של קבצים. חוטי ביצוע מאפשרים פעולות אחרות בזמן המתנה.

15.2 בטיחות חוטים

בטיחות חוטים (thread safety) קריטית: הגנה על משתנים משותפים באמצעות מנעולים, שימוש ב-`queue.Queue` להעברת מידע, הימנעות מנעילה הדדית, ושימוש ב--context man agers.

15.3 רשימת בדיקה: עיבוד מקבילי

1. **זיהוי פעולות:** זיהוי פעולות I/O-bound/CPU-bound, בחירת כלי נכון, הערכת תועלת
2. **יישום:** מספר תהליכים/חוטים דינמי, שיתוף נתונים בטוח, סנכרון נכון
3. **ניהול משאבים:** סגירה נכונה, טיפול בחריגות, מניעת זליגת זיכרון
4. **בטיחות:** הגנה על משתנים משותפים, מניעת מצבי תחרות ונעילה הדדית

כל הזכויות שמורות - Dr. Segal Yoram ©

16 עיצוב מודולרי ואבני בניין

עיצוב מבוסס אבני בניין (building blocks design) הוא גישה מודולרית בה כל רכיב הוא יחידה עצמאית עם ממשק מוגדר היטב.

16.1 מבנה אבן בניה

כל אבן בניה מוגדרת על ידי:

- **נתוני קלט** (Input Data) — סוגי נתונים, תחום תקף, תלויות חיצוניות, ולידציה מקיפה
- **נתוני פלט** (Output Data) — סוגי נתונים, פורמט, התנהגות במקרי קצה
- **נתוני הגדרה** (Setup Data) — פרמטרים עם ברירות מחדל, קונפיגורציה, אתחול

16.2 עקרונות עיצוב

אחריות יחידה (Single Responsibility) — כל אבן בניה אחראית למשימה אחת. **הפרדת דאגות** (Separation of Concerns) — כל אבן עוסקת בהיבט אחד. **שימוש חוזר** (Reusability) — אבני בניה עצמאיות ולא תלויות בקוד ספציפי. **יכולת בדיקה** (Testability) — כל אבן ניתנת לבדיקה באמצעות הזרקת תלות.

16.3 דוגמה לאבן בניה

Building Block Example

```
class DataProcessor:
    """
    Input:  raw_data (List[Dict]),
            filter_criteria (Dict)
    Output: processed_data (List[Dict])
    Setup:  processing_mode ('fast'/'accurate'),
            batch_size (int, default: 100)
    """

    def __init__(self, processing_mode='fast',
                 batch_size=100):
        self.processing_mode = processing_mode
        self.batch_size = batch_size
        self._validate_config()

    def process(self, raw_data, filter_criteria):
        self._validate_input(raw_data,
                             filter_criteria)
        return self._do_processing(
            raw_data, filter_criteria)

    def _validate_config(self):
```

```
        if self.processing_mode \
                not in ['fast', 'accurate']:
            raise ValueError("Invalid mode")
        if self.batch_size <= 0:
            raise ValueError("Batch size > 0")

    def _validate_input(self, data, criteria):
        if not isinstance(data, list):
            raise TypeError("data must be list")
        if not isinstance(criteria, dict):
            raise TypeError("criteria must be dict")
```


© Dr. Segal Yoram - כל הזכויות שמורות

17 רשימת בדיקה סופית

לפני הגשת הפרויקט, יש לעבור על רשימת בדיקה מקיפה.

17.1 מבנה ותיעוד חובה

- `README.md` מקיף בשורש הפרויקט ברמת מדריך משתמש
- תיקיית `docs/` עם `PRD.md`, `PLAN.md` ו-`TODO.md`
- מסמכי PRD ייעודיים לכל אלגוריתם/מנגנון מרכזי
- תיעוד ארכיטכטורה עם תרשימים ברורים
- ספר פרומפטים מתועד

17.2 ארכיטקטורה וקוד

- ארכיטקטורת SDK — כל הלוגיקה העסקית דרך שכבת SDK
- עיצוב OOP — אין כפילות קוד, שימוש בירושה ו-mixins
- שומר סף API — כל הקריאות החיצוניות דרך Gatekeeper
- הגבלות קצב מקונפיגורציה, ניהול תורות לגלישה
- קבצים עד 150 שורות קוד, הערות ו-docstrings
- עקביות בסגנון קוד, שמות תיאוריים

17.3 בדיקות ואיכות

- TDD — בדיקות נכתבות לפני/עם הקוד
- כיסוי בדיקות 85% ומעלה
- אפס הפרות Ruff
- תיעוד מקרי קיצון וטיפול בשגיאות
- דוחות בדיקה אוטומטיים

17.4 קונפיגורציה ואבטחה

- קבצי קונפיגורציה נפרדים מהקוד עם גרסאות
- `.env-example` עם ערכי דמה
- אין מפתחות API או סודות בקוד
- `.gitignore` מעודכן
- שימוש ב-uv כמנהל חבילות יחיד
- `pyproject.toml` ו-`uv.lock` קיימים

17.5 מחקר וויזואליזציה

- ניסויים שיטתיים עם שינוי פרמטרים
- ניתוח רגישות מתועד, מחברת ניתוח עם גרפים
- גרפים איכותיים, צילומי מסך, תרשימי ארכיטקטורה
- ניתוח עלויות tokens ואסטרטגיות אופטימיזציה

17.6 הרחבה ותקנים

- נקודות הרחבה (extension points) מתועדות
- ארגון כחבילת Python מקצועית
- עיבוד מקבילי עם בטיחות חוטים
- עיצוב מבוסס אבני בניין
- עמידה בתקן ISO/IEC 25010
- היסטוריית Git מסודרת, רישיון, ייחוס, הוראות deployment

כל הזכויות שמורות - Dr. Segal Yoram ©

18 מקורות ותקנים נוספים

לצורך הכנת פרויקט ברמת מצוינות, מומלץ להתייחס לתקנים ומקורות בינלאומיים: תכנית אבטחת איכות התוכנה של MIT[16], מודל איכות התוכנה ISO/IEC 25010[15], שיטות העבודה ההנדסיות של Google[17], ההנחיות ל-API של Microsoft[18], וההיוריסטיקות לשימושיות של נילסן[14].

19 הערה חשובה

מסמך זה מציג רמת מצוינות גבוהה במיוחד. לא כל סעיף הוא מחויב במלואו, אך ככל שיותר קריטריונים מתקיימים, כך הערכת האיכות תהיה גבוהה יותר. התמקדו בעומק, במקצועיות ובהדגמת יכולות פיתוח ברמה גבוהה.

מומלץ להשתמש בכלי LLM וסוכני AI לעזרה בהשלמת הפרויקט. מובהר כי כחלק מהבדיקה יתכן וייעשה שימוש בסוכני AI לביצוע הבדיקה.

19.1 כרטיס עזר מהיר לדרישות

טבלה 5: סיכום דרישות איכות קוד

אכיפה	סף	כלל
סקירת קוד	כל הלוגיקה דרך SDK	ארכיטקטורת SDK
סקירת קוד	חילוץ ב-2+ עותקים	OOP / ללא כפילות
סקירת קוד + בדיקה	כל הקריאות דרכו	שומר סף API
בדיקת קונפיגורציה	מקונפיגורציה, לא מקוד	הגבלות קצב
בדיקת אינטגרציה	תור, לא קריסה	ניהול גלישה
מודול גרסה	מתחיל ב-1.00	בקרת גרסאות
תהליך עבודה	אדום-ירוק-שיפור	TDD
בדיקה אוטומטית	$\leq$ 150 שורות	גודל קבצים
ruff check	0 הפרות	Linter
pytest –cov	$\geq$ 85%	כיסוי בדיקות
סקירת קוד	0 בקוד מקור	ערכים מוטבעים
סריקה אוטומטית	0 + .env-example	סודות
בדיקה אוטומטית	הכל דרך uv	מנהל חבילות

כל הזכויות שמורות - Dr. Segal Yoram ©

English References

1 Archbee, *Readme files guide*, https://www.archbee.com/blog/readme-files-guide, 2024.

2 GitHub, *About readmes*, https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes, 2024.

3 Monday.com, *Prd template - product requirement document*, https://monday.com/blog/rnd/prd-template-product-requirement-document/, 2024.

4 Miro, *Modular prd template*, https://miro.com/templates/modular-prd/, 2024.

5 Aha! *What is a good product requirements document template*, https://www.aha.io/roadmapping/guide/requirements-management, 2024.

6 Daily.dev, *10 code commenting best practices for developers*, https://daily.dev/blog/10-code-commenting-best-practices-for-developers, 2024.

7 Stack Overflow, *Best practices for writing code comments*, https://stackoverflow.blog/2021/12/23/best-practices-for-writing-code-comments/, 2021.

8 Codacy, *Code documentation best practices*, https://blog.codacy.com/code-documentation, 2024.

9 Hoop.dev, *Api security best practices: Protecting secrets with environment variables*, https://hoop.dev/blog/api-security-best-practices-protecting-secrets-with-environment-variables/, 2024.

10 Claude Support, *Api key best practices: Keeping your keys safe and secure*, https://support.claude.com/en/articles/9767949-api-key-best-practices, 2024.

11 OpenAI, *Best practices for api key safety*, https://help.openai.com/en/articles/5112595-best-practices-for-api-key-safety, 2024.

12 UX Design, *Measuring design quality with heuristics*, https://uxdesign.cc/measuring-design-quality-with-heuristics-44857efa514, 2024.

13 J. Nielsen, *10 usability heuristics for user interface design*, https://www.nngroup.com/articles/ten-usability-heuristics/, 1994.

14 J. Nielsen, *10 usability heuristics for user interface design*, https://www.nngroup.com/articles/ten-usability-heuristics/, 1994.

15 ISO, *Iso/iec 25010:2011 systems and software quality requirements and evaluation*, https://www.iso.org/standard/35733.html, 2011.

16 MIT ACIS, *Mit software quality assurance plan*, https://acisweb.mit.edu/acis/sqap/sqap.r1.html, 2022.

17 Google, *Google engineering practices documentation*, https://google.github.io/eng-practices/, 2023.

18 Microsoft, *Microsoft rest api guidelines*, https://github.com/microsoft/api-guidelines, 2023.

 © Dr. Segal Yoram - כל הזכויות שמורות

20 נספח: הנחיות מפורטות להגשת תוכנה מקצועית

נספח זה מרכז את כלל ההנחיות המפורטות להגשת פרויקט תוכנה מקצועי. ההנחיות מאורגנות בצורה שיטתית ומכסות את כל ההיבטים הנדרשים.

20.1 מסמכי פרויקט ותכנון

מסמך דרישות מוצר (PRD):

- סקירת הפרויקט והקשר — תיאור מטרת הפרויקט, בעיית המשתמש, ניתוח שוק וזיהוי קהל יעד
- יעדים ומדדי הצלחה — יעדים מדידים, מדדי KPI, קריטריוני קבלה
- דרישות פונקציונליות ולא-פונקציונליות — רשימת תכונות, user stories, דרישות ביצועים ואבטחה
- הנחות, תלויות ומגבלות — מערכות חיצוניות, מגבלות טכניות, פריטים מחוץ לתחום
- ציר זמן ואבני דרך — לוח זמנים מפורט עם נקודות ביקורת ומשלוחים

מסמך ארכיטקטורה:

- תרשימי C4 Model (Context, Container, Component, Code)
- תרשימי UML לתהליכים מורכבים
- תרשימי deployment וארכיטקטורה תפעולית
- החלטות ארכיטקטוניות (ADRs) עם רציונל ו-trade-offs
- תיעוד API וממשקים, סכימות נתונים וקונטרקטים

20.2 תיעוד קוד ומבנה פרויקט

קובץ README מקיף:

- הוראות התקנה — דרישות מערכת, התקנה שלב-אחר-שלב, הגדרת משתני סביבה, פתרון בעיות
- הוראות הפעלה — הרצה במצבים שונים, דגלים ואפשרויות, workflow טיפוסי
- דוגמאות והדגמות — דוגמאות קוד, צילומי מסך, תרחישי שימוש, קישורים לסרטונים
- מדריך תצורה — קבצי קונפיגורציה, פרמטרים והשפעתם
- הנחיות תרומת קוד — תקני קוד וסגנון
- רישיון וייחוס — רישיון שימוש, ייחוס לספריות צד שלישי

מבנה פרויקט מודולרי:

- חלוקה לתיקיות לפי תפקיד: `src/`, `tests/`, `docs/`, `data/`, `results/`, `config/`, `assets/`
- קבצים עד כ-150 שורות קוד
- שמות תיקיות וקבצים תיאוריים ועקביים

- הפרדת אחריות ברורה

איכות קוד:

- הערות המסבירות "למה" ולא רק "מה"
- Docstrings לכל פונקציה, מחלקה ומודול
- שמות משתנים ופונקציות תיאוריים
- פונקציות קצרות בגישת אחריות יחידה
- עקרון DRY — ללא כפילות קוד

20.3 ניהול קונפיגורציה ואבטחת מידע

קבצי קונפיגורציה:

- קבצים נפרדים בפורמט `.json`, `.yaml` או `.env`
- ללא ערכים מוטבעים בקוד
- קבצי דוגמה (`.env-example`) עם ברירות מחדל
- `.gitignore` למניעת העלאת קבצים רגישים
- קבצי template לסביבות שונות

אבטחת מידע:

- איסור מוחלט על שמירת מפתחות API בקוד
- שימוש במשתני סביבה בלבד
- הסתרת `.env` באמצעות `.gitignore`
- החלפת מפתחות תקופתית וניטור שימוש
- הגבלת הרשאות למינימום (least privilege)

20.4 בדיקות ואיכות תוכנה

בדיקות יחידה:

- כיסוי מינימלי של 85% לקוד חדש
- כיסוי מוגבר לקוד קריטי ולוגיקה עסקית
- כיסוי הצהרות, ענפים ומסלולים קריטיים
- אוטומציה ב-CI/CD pipeline
- דוחות כיסוי

טיפול במקרי קיצון:

- זיהוי שיטתי של תנאי גבול
- תיעוד כל מקרה קיצון עם קלט ותגובה צפויים
- תכנות הגנתי עם בדיקות קלט
- הודעות שגיאה ברורות ולוגים מפורטים

כל הזכויות שמורות - Dr. Segal Yoram ©

- הידרדרות חלקה במקרי כשל

20.5 מחקר וניתוח

חקר פרמטרים:

- ניסויים שיטתיים עם שינוי פרמטרים מבוקר
- שיטות ניתוח: נגזרות חלקיות, ניתוח שונות, OAT
- טבלת ניסויים, גרפים ממחישים, ניתוח סטטיסטי

ויזואליזציה:

- Bar charts, Line charts, Scatter plots, Heatmaps, Box plots
- תוויות ברורות, צבעים עקביים ונגישים, רזולוציה גבוהה
- כלים: Matplotlib, Seaborn, Plotly, Tableau, D3.js

20.6 ממשק משתמש

- קריטריוני שימושיות: למידה, יעילות, זכירות, מניעת שגיאות, שביעות רצון
- עשרת ההיוריסטיקות של נילסן
- צילומי מסך של כל מסך ומצב
- תיאור workflow של משתמש
- שיקולי נגישות

20.7 ניהול גרסאות ועלויות

ניהול גרסאות:

- היסטוריית commits ברורה עם הודעות משמעותיות
- branches נפרדים לתכונות חדשות
- Pull Requests עם סקירות קוד
- tagging לגרסאות מרכזיות
- ספר פרומפטים מתועד

ניתוח עלויות:

- ספירת tokens בכניסה ויציאה
- חישוב עלות למיליון tokens לפי מודל
- תחזית עלויות, ניטור בזמן אמת, התראות תקציב
- אסטרטגיות אופטימיזציה

20.8 הרחבה ותקנים

- ארכיטקטורת תוספים עם ממשקים ברורים
- נקודות חיבור (hooks), middleware, עיצוב API-first
- מודולריות, שימוש חוזר, ניתנות לניתוח ובדיקה
- עמידה בתקן ISO/IEC 25010
- הפניה לתקנים בינלאומיים: MIT SQA, Google Engineering, Microsoft API, Nielsen

20.9 רשימת בדיקה סופית

1. תיעוד: PRD, ארכיטקטורה, README, תיעוד API, ספר פרומפטים
2. קוד: מבנה מודולרי, קבצים $\leq$ 150 שורות, הערות ו-docstrings, עקביות סגנון
3. קונפיגורציה: קבצים נפרדים, `.env-example`, ללא סודות, `.gitignore`
4. בדיקות: כיסוי 85%+, מקרי קיצון, טיפול בשגיאות, דוחות אוטומטיים
5. מחקר: חקר פרמטרים, ניתוח רגישות, מחברת ניתוח, גרפים
6. ויזואליזציה: גרפים איכותיים, צילומי מסך, תרשימי ארכיטקטורה
7. עלויות: טבלת tokens, ניתוח מפורט, אופטימיזציה
8. הרחבה: נקודות הרחבה, דוגמאות תוספים, ממשקים
9. כלליים: Git history, רישיון, ייחוס, deployment

כל הזכויות שמורות - Dr. Segal Yoram ©